

11th Progress Report

W. Chan

Date:	15/04/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- Went through the timeline of attempt at implementing eligibility traces, talked about weird things observed in testing (wrong auto-diff works better, hard to replicate elevator reversal results despite using identical short-period dynamics, difficult hyperparameter tuning, ...)
- Discussed possible results to gather, what flight scenarios + algorithm combinations to run, and what hyperparameters to use:
 1. Last report showed a 6 by 4 table of possible algorithm and flight scenario combinations. Most definitely too many combinations to analyze comprehensively, thus there is no need to go over every single one in the thesis. Should only cover the most performant and least performant augmentations (possibly vanilla IDHP too if that's not in either camp), this is enough information to observe the effect of different augmentations on the performance of IDHP (performance in terms of episode return, convergence rate, and maybe even PSD error).
 2. Regarding what hyperparameters to use per combination, while it would be unfair to use the same parameters for all combinations, it would also be too time-intensive to find optimal parameters for each combination. A possible compromise would be to dedicate the same amount of effort to tuning each combination. Another possible compromise is to test run each combination on the same set of hyperparameters, where this set of parameters would span at least a bit of the hyperparameter space.
 3. Possibly also present/discuss the most optimal performance found for some of the combinations if extra-ordinary performances are observed (super good return, super low divergence rate...). E.g., if replacing traces shows extremely good convergence compared to vanilla IDHP on cg shift fault scenario, this could be mentioned in some way: "it was observed that augmenting IDHP with replacing traces allowed its convergence rate to dramatically improve over the baseline case..."
- Did not have time to go over:
 1. Flying V vs citation for thesis
 2. New research objectives.
 3. Literature review.

Description of the results since last meeting

- Iterated on the research objective and question:

Research Objective

To advance the state-of-the-art reinforcement learning based flight controllers and improve their fault tolerance, by researching novel augmentations to online reinforcement learning algorithms.

Research Questions

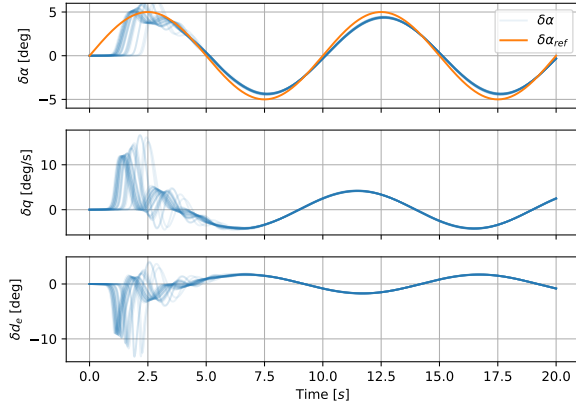
1. What reinforcement learning algorithm can yield a flight controller which is the most tolerant to faults?
 - (a) What RL algorithms are considered to be state-of-the-art?
 - (b) How is fault tolerance defined and tested in past research?
 - (c) Which algorithms have been shown to provide the best fault tolerance?
 - (d) What reference tracking performance have these algorithms shown in past research?
 - (e) What promising augmentations to reinforcement learning algorithms can be made and experimented with?
2. How can the identified algorithm be applied to control the PH-LAB research aircraft?
 - (a) How can the identified augmentations be made to the reinforcement learning algorithm studied?
 - (b) How should the flight control system be structured?
 - (c) What are the variables defining the MDP in the case of controlling the PH-LAB?
3. How does the implemented flight controller perform during nominal flight and in the presence of faults?
 - (a) What flight scenarios should be designed to test the performance and fault tolerance of the controller?
 - (b) What is the nominal performance and degree of fault tolerance of the implemented controller?
 - (c) How do the proposed augmentations affect the flight control characteristics of the reinforcement learning algorithm?
 - (d) How well does the nominal and faulty flight performance of the proposed flight controller compare to a traditional PID controller?

- 6 (augmented?) algorithms currently exist:
 1. IDHP,
 2. IDHP replacing trace (IDHP-RT),
 3. IDHP accumulating trace (IDHP-AT),
 4. Multistep IDHP (MIDHP),
 5. Multistep IDHP replacing trace (MIDHP-RT),
 6. Multistep IDHP accumulating trace (MIDHP-AT).

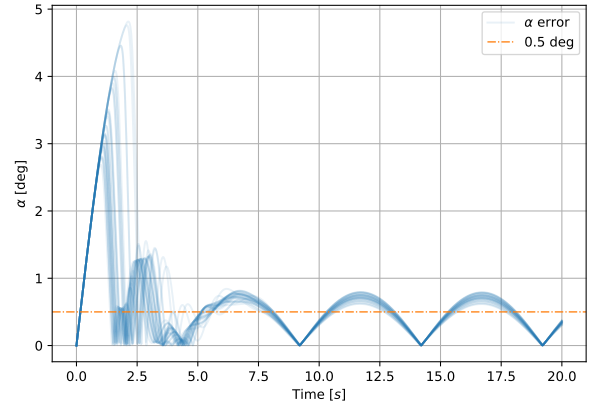
- Implemented replacing trace version of eligibility trace according to Equation 1 and verified it works (checking if the eligibility trace gets overwritten if the norm of gradient is greater).

$$\mathbf{E}(t) = \begin{cases} \nabla f(\mathbf{x}) & \text{if } \|\nabla f(\mathbf{x})\| > \|\mathbf{E}(t-1)\| \\ \lambda\gamma\mathbf{E}(t-1) & \text{otherwise} \end{cases} \quad (1)$$

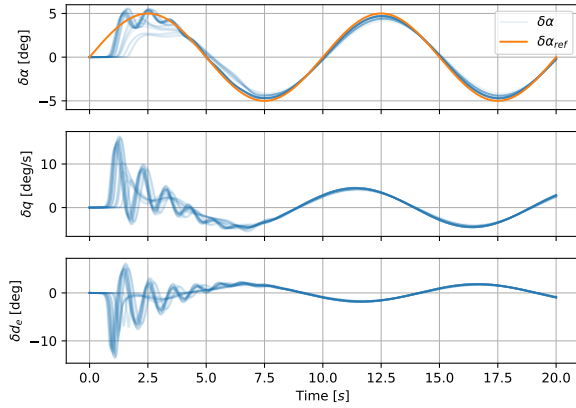
Preliminary performance seems a little better than IDHP, but worse than accumulating traces. A comparison of IDHP and IDHP-RT is shown in Figure 1.



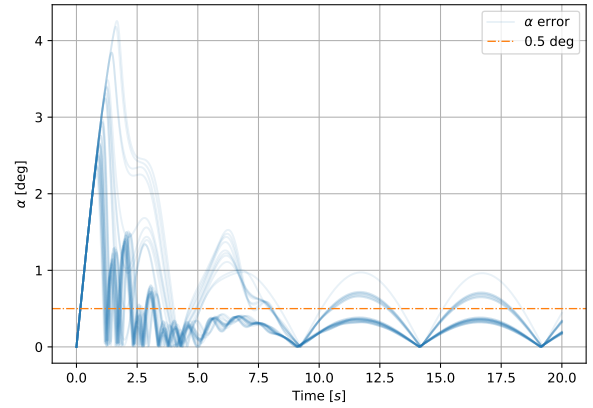
(a) IDHP state action time trace.



(b) IDHP α error time trace.



(c) IDHP-RT state action time trace.



(d) IDHP- α error time trace.

Figure 1: Comparison of state action and α error time traces between IDHP and IDHP-RT, average episode return for IDHP and IDHP-RT are -0.0869 and -0.0829 respectively.

- Implemented MIDHP according to the following formulation for the critic update Equation 2 and actor update Equation 3.

$$\mathbf{w}_C(t) = \mathbf{w}_C(t-1) - \eta_c \frac{\partial E_2(t-1)}{\partial \mathbf{w}_C(t-1)} \quad (2)$$

Where
$$\frac{\partial E_2(t)}{\partial \mathbf{w}_C(t)} = \frac{\partial E_2(t)}{\partial \mathbf{e}(t)} \frac{\partial \mathbf{e}(t)}{\partial \lambda(\mathbf{x}_t)} \frac{\partial \lambda(\mathbf{x}_t)}{\partial \mathbf{w}_C(t)}$$

$$= \frac{1}{2} \mathbf{e}_2(t)^\top \frac{\partial \lambda(\mathbf{x}_t)}{\partial \mathbf{w}_C(t)}$$

$$\mathbf{e}_2(t) = \lambda(\mathbf{x}_{t-2})|_{t-2} - \frac{\partial c_{t-2}}{\partial \mathbf{x}_{t-2}}|_{t-2} - \gamma \frac{\partial c_{t-1}}{\partial \mathbf{x}_{t-1}}|_{t-1} \frac{\partial \mathbf{x}_{t-1}}{\partial \mathbf{x}_{t-2}}|_{t-2} - \gamma^2 \lambda'(\mathbf{x}_t) \frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_{t-1}}|_{t-1} \frac{\partial \mathbf{x}_{t-1}}{\partial \mathbf{x}_{t-2}}|_{t-2}$$

$$\mathbf{w}_A(t) = \mathbf{w}_A(t-1) - \eta_a \frac{\partial L_2(t-1)}{\partial \mathbf{w}_A(t-1)} \quad (3)$$

Where

$$\frac{\partial L_2(t)}{\partial \mathbf{w}_A(t)} = \frac{\partial L_2(t)}{\partial \mathbf{x}(t+1)} \frac{\partial \mathbf{x}(t+1)}{\partial \mathbf{a}(t)} \frac{\partial \mathbf{a}(t)}{\partial \mathbf{w}_A(t)}$$

$$\frac{\partial L_2(t)}{\partial \mathbf{w}_A(t)} = \underbrace{\left[\frac{\partial c_t}{\partial \mathbf{x}_{t+1}} + \gamma \frac{\partial c_{t+1}}{\partial \mathbf{x}_{t+1}} + \gamma^2 \lambda'(\mathbf{x}_{t+2}) \frac{\partial \mathbf{x}_{t+2}}{\partial \mathbf{x}_{t+1}} \right]_{t+1}}_{\text{partial of } L_2(t) \text{ w.r.t. } \mathbf{x}(t+1)} \frac{\partial \mathbf{x}(t+1)}{\partial \mathbf{a}(t)} \bigg|_t \frac{\partial \pi(t, \mathbf{w}_A(t))}{\partial \mathbf{w}_A(t)}$$

Where λ' is the target critic network, whose weights are a moving average of the weights from the critic network λ , furthermore the reward gradient term $\frac{\partial c_t}{\partial \mathbf{x}_{t+1}}$ has been modelled as $\frac{\partial c_{t+1}}{\partial \mathbf{x}_{t+1}}$ under the assumption of a smooth and high sample rate system.

Comparison of the multistep and single-step versions of IDHP shows that the augmentation has actually slowed down convergence of IDHP, and can be seen in Figure 2.

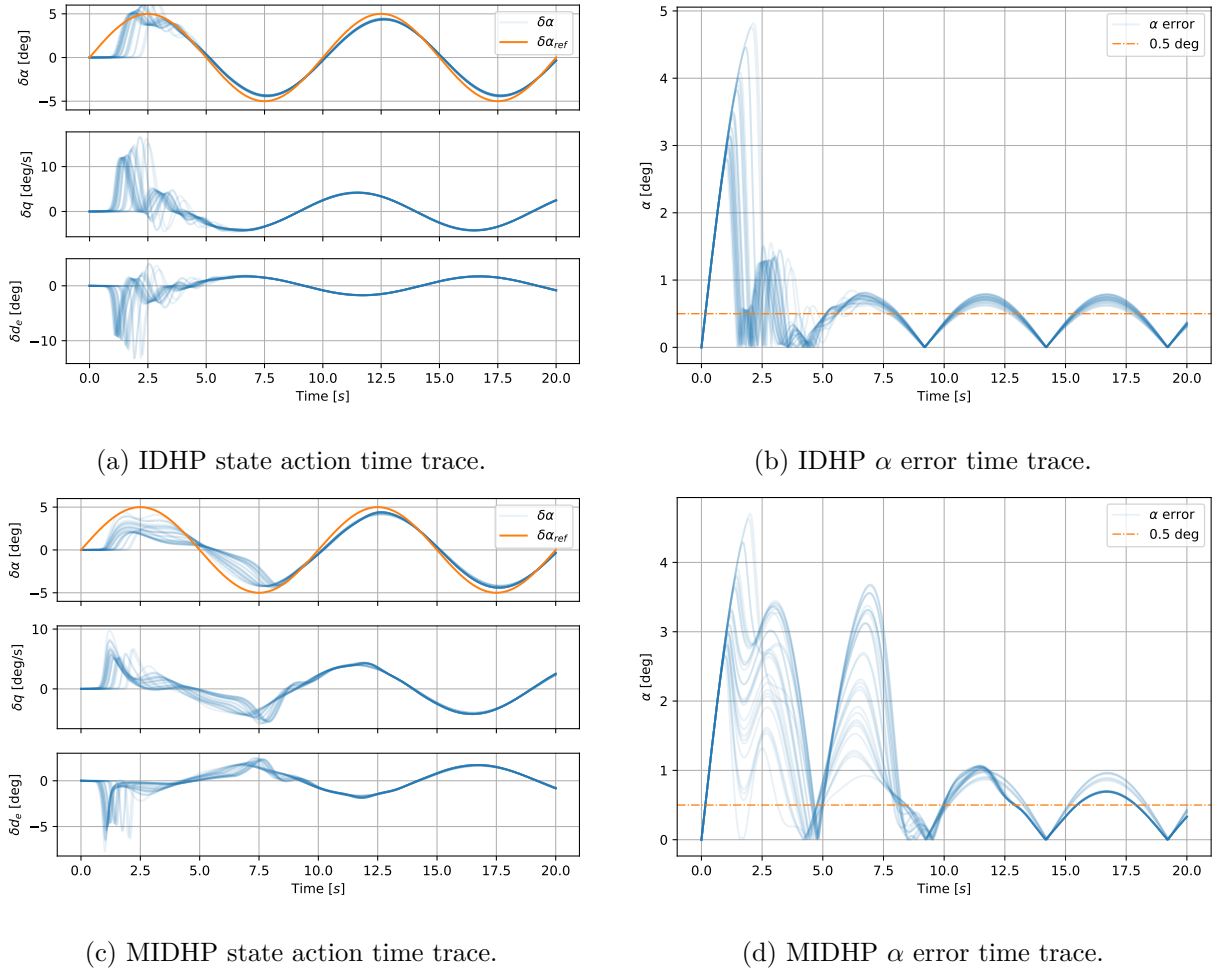


Figure 2: Comparison of state action and α error time traces between IDHP and MIDHP, average episode return for IDHP and MIDHP are -0.0869 and -0.237 respectively.

- **Optimization-problemification (bad name) of the hyperparameter tuning problem:**

The current approach to hyperparameter tuning is based on two empirical observations.

1. Elevator reversal fault is the most difficult scenario.

2. CG shift case constrains the hyperparameter space to 0 low (steady state) learning rate for critic, i.e. critic parameters cannot change after initial warmup or fault adaptation period.

By taking into account the second observation, the hyperparameters of all algorithms are tuned for the elevator reversal fault, and the best-performing parameters are used as the de facto algorithm setting for all other flight scenarios.

The tuning procedure as of now is naive and manual, it is possible that using an optimization algorithm for tuning will save on effort and even get more optimal results, and it is foreseeable that there is time left over in research phase 2 to do this.

The first avenue would probably be using off-the-shelf constrained 0th order optimizers from Scipy, I reckon using gradient-based optimizers is a bad idea since manual and naive tuning has thus far shown the hyperparameter space is highly nonconvex and multi-modal.

The second avenue would be to implement my own solver, in my internship I've already experimented with constrained evolutionary algorithms which could work if the constraint-satisfying hyperparameter space is a convex space (which it is as the constraints are simply inequalities on hyperparameters).

I have already figured out how to use multi-processing to get the metrics for one set of hyperparameters in parallel, so doing this is actually not too far out of reach, only considerations now are **a)** run-time, **b)** the fact that this foray is only a periphery topic to the present research, which **c)** makes implementation time for this foray relatively more expensive than the core implementations (figuring out slx to c to python for citation, the swig library, verifying the citation module, making the module for various flight conditions...), and finally **d)** it's not guaranteed to give better hyperparameters than manual tuning (though it does mean I don't need to manually select parameters and run the code for tuning).

The optimization problem for hyperparameter tuning would simply be :

$$\begin{aligned} & \underset{\mathbf{x}}{\operatorname{argmin}} && f(\mathbf{x}) \\ & \text{subject to} && \mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub} \end{aligned}$$

Where $\mathbf{x}$ is the set of hyperparameters, $f(\mathbf{x})$ is some function returning the “goodness” of the hyperparameters (e.g episode return of IDHP using $\mathbf{x}$), $\mathbf{lb}$ and $\mathbf{ub}$ being some upper and lower bounds for the hyperparameters.

- **Hyperparameter tuning results, comparison of 6 algorithms:**

Each hyperparameter is ran 30 times with different RNG seeds, each run initiates the actor and critic networks with a different set of random weights (this is the only randomness in the entire simulation). For one set of hyperparameter runs, it is characterized by 4 metrics:

1. Average episode return.
2. Average PSD error between response and reference α .
3. Average convergence time.
4. Number of runs with a convergence time greater than 30 seconds (equivalent to 10 s after fault triggered).

Where convergence is defined as the time when the α error settles below 0.5 degrees.

An example run is shown in Figure 3 and it's state action time trace, α error time trace, and PSD spectrogram. The time at which the elevator reversal fault being triggered is shown in a vertical solid black line, and the converged time is shown in a vertical dash-dot black line.

To compute the PSD error, the difference between reference and response PSD is taken at all sample frequencies and summed.

Compiling these 4 metrics, the performance of each hyperparameter can be compared against one another. Figure 4 plots 3 of the 4 metrics for all 6 possible algorithms on the elevator reversal fault, with the episode return metric omitted as **a)** its information is encapsulated in the PSD error metric already and **b)** for runs where convergence did not occur the return is excessively low leading to poor data comprehensibility if plotted.

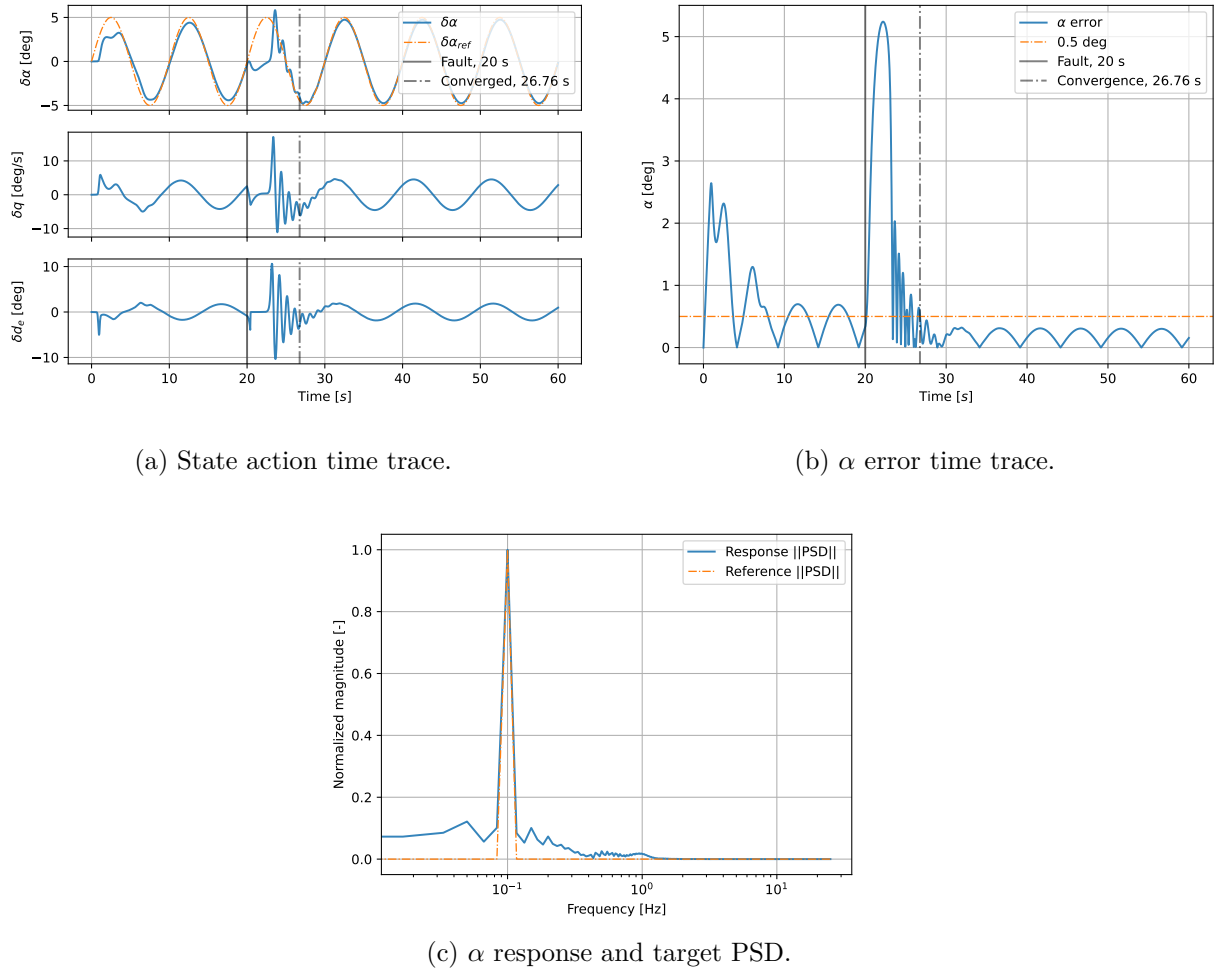


Figure 3: Example of one MIDHP-RT run with the convergence time labelled, PSD error is defined as the sum of the difference between response and reference PSD's at all sampled frequencies.

Plans for next 2 weeks

1. Gather results by running the best-found hyperparameters on the 4 flight cases (nominal, reversed elevator, damaged elevator, shifted cg) with more RNG seeds.
2. Begin writing in the mid-term report, probably parts of the introduction, of the methodology/theory, and results.
3. Revise literature review and fix all errors.
4. Refactor and make prettier code?
5. Am on track according to the Gantt chart, see Figure 5, still have 1 more week allocated for implementing MIDHP or other auxiliary technical stuff, however, will move up the start date of report writing to give myself more time to write the mid-term report.

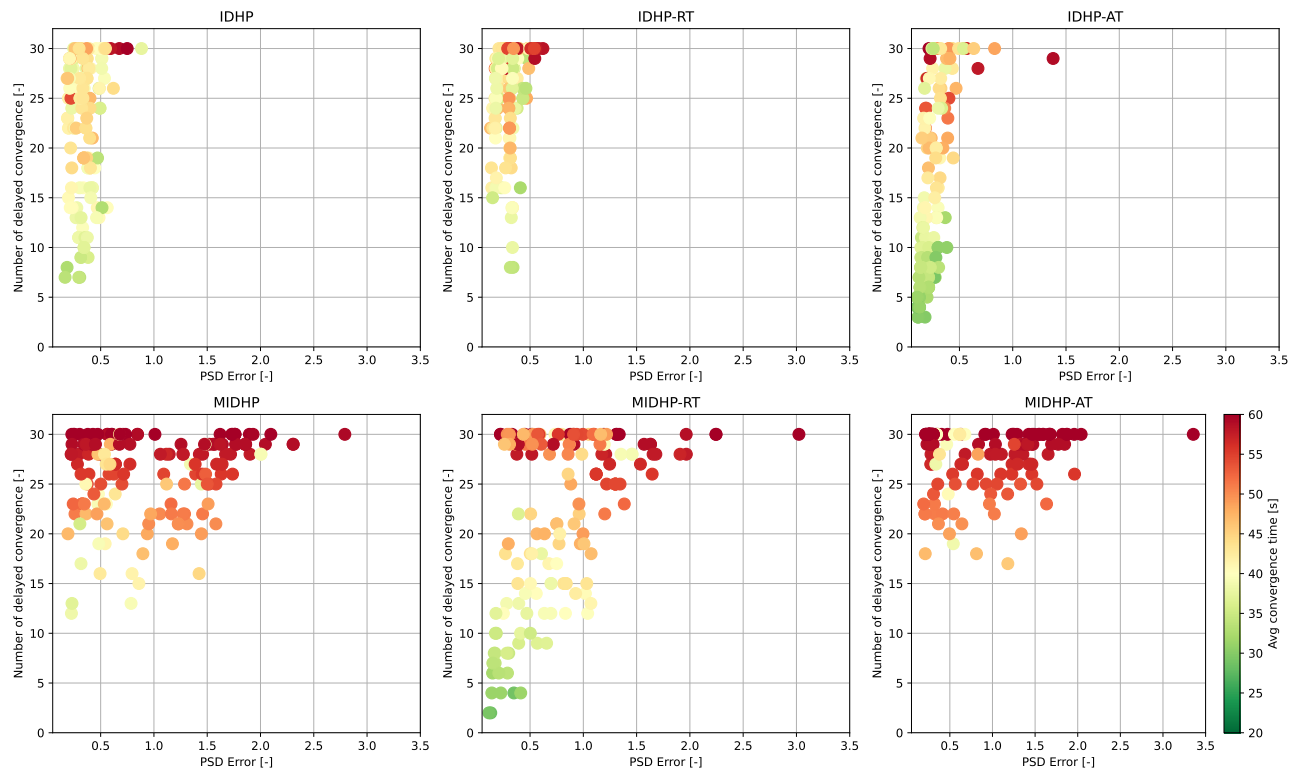


Figure 4: Scatter plots of metrics for 150 hyperparameters: average PSD error, number of runs with convergence time greater than 30 s (10 s after fault), and average convergence time.

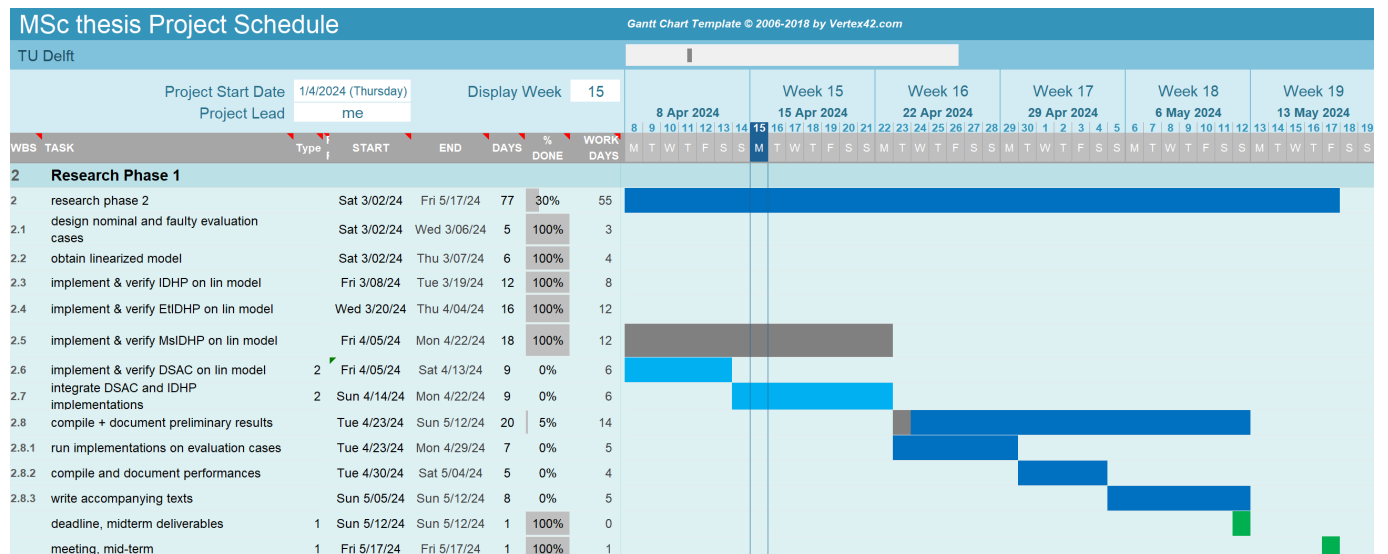


Figure 5: Gantt chart of research phase one.